

✅ Where each feature lives now (your exact mapping)

Extraction: `services/extraction_service.py`
(`extract_text_from_file`)

Text preprocessing: `utils/text_utils.py`

Chunking: `utils/chunking.py`

Embeddings generation: `services/embedding_service.py` + model
init in `models/embedding_model.py`

Semantic retrieval: `services/retrieval_service.py`

Summarization (and fallback):
`services/summarization_service.py`

Entity extraction: `utils/entity_extraction.py`

Translation + “force language”: `services/translation_service.py`

QA chatbot: `services/qa_service.py`

MongoDB connection/utils: `database/mongo_connection.py`

In-memory document storage: `storage/document_store.py`

Session/token storage: `utils/security_utils.py`

Routes

Auth: `routes/auth_routes.py`

Upload + summary + history + document info + health:
`routes/document_routes.py`

Chatbot endpoint: `routes/qa_routes.py`

Dataset upload: `routes/dataset_routes.py`

✅ Data-flow (end-to-end)

Upload & summarize (POST /api/upload_summary)

routes/document_routes.py

- services/extraction_service.extract_text_from_file()
- services/summarization_service.generate_summary() (uses OpenAI or fallback)
- utils/chunking.split_into_chunks()
- services/embedding_service.generate_embeddings()
- storage/document_store.store_document() returns documentId
- optionally writes history to Mongo via database/mongo_connection.get_documents_collection()

Ask question (POST /api/ask)

routes/qa_routes.py

- storage/document_store.get_document(doc_id)
- services/qa_service.answer_question_with_context()
- services/retrieval_service.retrieve_relevant_chunks() (semantic search)
- OpenAI answer (strictly from context) or fallback extractive answer
- services/translation_service.force_language()

Dataset upload (POST /api/upload_dataset)

routes/dataset_routes.py

→ parse CSV/TSV

→ per row: services/summarization_service.generate_summary()

→ chunk + embed + storage/document_store.store_document()

→ return train/test ids